

Now, let us examine a quantum program that demonstrates the effect of a Hadamard gate on a qubit through measurement. Consider the program, titled '*qiskit3.py*' (with line numbers included for clarity), shown below. This program applies a Hadamard gate to a qubit and then measures its state to illustrate the outcome of the operation.

```
1. from qiskit import QuantumCircuit
2. from qiskit_aer import AerSimulator

3. qc = QuantumCircuit(1, 1)
4. qc.h(0)
5. qc.measure(0, 0)

6. sim = AerSimulator( )

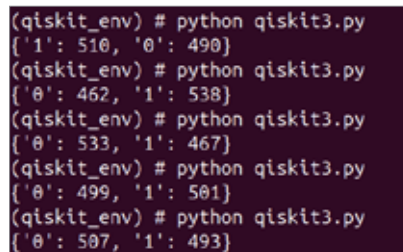
7. result = sim.run(qc, shots=1000).
  result( )
8. counts = result.get_counts( )
9. print(counts)
```

Now let us understand how the program works. Lines 1, 3, 4, and 5 are identical to those in '*qiskit2.py*' and therefore require no further explanation. Line 2 imports the class *AerSimulator* from the package *qiskit_aer*. The *AerSimulator* enables us to execute quantum circuits on a classical computer and observe their measurement outcomes. Before proceeding, ensure that the *AerSimulator* has been installed. If you have not already done so, it can be installed using the command '*pip install qiskit-aer*'.

Line 6 creates an instance of the *AerSimulator* and assigns it to the variable *sim*. This simulator is then used to execute the quantum circuit. Line 7 runs the circuit on the simulator. The argument '*shots=1000*' instructs the simulator to execute the circuit 1,000 times. A large number of shots is necessary because a single measurement reveals only one outcome—either 0 or 1. By repeating the experiment many times, we can estimate the probabilities associated with the possible outcomes. The method *result()* returns an

object containing the results of all these executions. Line 8 extracts the measurement statistics from the *result* object and stores them in the variable *counts*. These statistics are returned as a dictionary in which the keys represent the measurement outcomes and the values indicate the number of times each outcome occurred. Line 9 displays the contents of the dictionary *counts*.

Since the qubit is in an equal superposition of $|0\rangle$ and $|1\rangle$, the outcomes 0 and 1 are expected to occur with approximately equal frequency. However, the counts will generally not be exactly equal because quantum measurement is inherently probabilistic. Unlike the previous examples, we will execute '*qiskit3.py*' multiple times to observe the probabilistic nature of quantum computing. Figure 2 shows the results of five successive executions of the program.



```
(qiskit_env) # python qiskit3.py
{'1': 510, '0': 490}
(qiskit_env) # python qiskit3.py
{'0': 462, '1': 538}
(qiskit_env) # python qiskit3.py
{'0': 533, '1': 467}
(qiskit_env) # python qiskit3.py
{'0': 499, '1': 501}
(qiskit_env) # python qiskit3.py
{'0': 507, '1': 493}
```

Figure 2: Five successive executions of the program *qiskit3.py*

Now, let us examine the output obtained from five successive executions of the program '*qiskit3.py*'. Recall that the Hadamard gate places the qubit in the state $|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ which implies that the probabilities of obtaining the outcomes 0 and 1 upon measurement are both 50%. Since the circuit is executed 1000 times (1000 shots), one might be tempted to expect exactly 500 occurrences of 0 and 500 occurrences of 1. However, quantum measurement is inherently probabilistic, and each measurement outcome is random. Consequently, the actual numbers fluctuate from one execution of the program to another, giving results such as 510–490, 462–538, or 499–501. These deviations are perfectly normal and are

analogous to the outcome of tossing a fair coin 1000 times, where one rarely obtains exactly 500 heads and 500 tails. As the number of shots increases, the measured frequencies tend to approach the theoretical probabilities predicted by quantum mechanics. It is important to note that these fluctuations are not caused by imperfections in the simulator. Rather, they are a manifestation of the intrinsic probabilistic nature of quantum measurement itself, one of the most fundamental features of quantum mechanics.

Now, let us try to understand how critical measurement is in quantum computing and how quantum mechanics plays a central role in the process. In a classical computer, reading a bit does not disturb its value. If a bit stores 0, reading it simply reveals 0 and the bit remains 0. Similarly, if the bit stores 1, reading it reveals 1 without changing the state of the bit.

A qubit behaves very differently. Consider the superposition state $|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. When this qubit is measured, the outcome is 0 with probability 1/2 and 1 with probability 1/2. More importantly, the act of measurement changes the state of the qubit itself. If the outcome is 0, the qubit collapses to the state $|0\rangle$; if the outcome is 1, it collapses to the state $|1\rangle$. This phenomenon, known as wavefunction collapse, is one of the most surprising features of quantum mechanics.

The collapse of the wavefunction is both a curse and a boon. It is a curse because measurement destroys the superposition and the original quantum state cannot be recovered. However, it is also a boon because measurement provides the classical information that constitutes the final output of a quantum computation. Thus, measurement acts as a bridge between the quantum world and the classical world.

Pauli Gates

As promised in the previous article, we shall continue our exploration of